Unit III

3. Multimedia, Animation and Graphics

3.1 Playing Audio, Video
3.2 Rotate Animation, FadeIn / FadeOut Animation
3.3 Zoom Animation, Scale animation
3.4 2D and 3D graphics

3.1 Playing Audio, Video:

Here's a concise explanation on **Playing Audio and Video in Android**:

1. Playing Audio:

Android uses the `**MediaPlayer**` class to play audio files (MP3, WAV, etc.).

Steps to Play Audio:

1. Place the audio file in the `res/raw/` folder.
2. Use `MediaPlayer` to load and play it.

java

```
MediaPlayer mediaPlayer = MediaPlayer.create(this, R.raw.sample_audio);
mediaPlayer.start(); // starts playing
```

You can also use methods like `.pause()`, `.stop()`, and `.release()` to control playback.

2. Playing Video:

For video, Android provides the `**VideoView**` class and `**MediaController**` for playback controls.

Steps to Play Video:

1. Add a `VideoView` in your layout.
2. Set the video URI and start playback.

java

```
VideoView videoView = findViewById(R.id.videoView);
videoView.setVideoURI(Uri.parse("android.resource://" + getPackageName() +
"/" + R.raw.sample_video));
videoView.setMediaController(new MediaController(this));
videoView.start();
```

Supports local and online video files (MP4, 3GP, etc.).

These classes make it easy to embed **multimedia playback** in Android apps with minimal code.

3.2 Rotate Animation, FadeIn / FadeOut Animation:

Here's a concise explanation on **Rotate Animation** and **FadeIn / FadeOut Animation** in Android:

1. Rotate Animation:

Used to rotate a view (e.g., image or button) around a pivot point.
Can be done in XML or programmatically.

XML Example (`res/anim/rotate.xml`):

xml

```
<rotate
    xmlns:android=http://schemas.android.com/apk/res/android
    android:fromDegrees="0"
    android:toDegrees="360"
    android:duration="1000"
    android:pivotX="50%"
    android:pivotY="50%" />
```

In Java:

java

```
Animation rotate = AnimationUtils.loadAnimation(this, R.anim.rotate);
imageView.startAnimation(rotate);
```

2. FadeIn / FadeOut Animation:

Used to smoothly change the **visibility** of a view by altering its **opacity (alpha)**.

Fade In (`fade_in.xml`):

xml

```
<alpha
    xmlns:android=http://schemas.android.com/apk/res/android
    android:fromAlpha="0.0"
    android:toAlpha="1.0"
    android:duration="1000" />
```

Fade Out (`fade_out.xml`):

xml

```
<alpha
    xmlns:android=http://schemas.android.com/apk/res/android
    android:fromAlpha="1.0"
    android:toAlpha="0.0"
    android:duration="1000" />
```

Apply in Java:

java

```
Animation fadeIn = AnimationUtils.loadAnimation(this, R.anim.fade_in);
textView.startAnimation(fadeIn);
```

These animations enhance user experience by adding smooth transitions to your app's UI.

3.3 Zoom Animation, Scale animation:

Here's a concise explanation on **Zoom Animation** and **Scale Animation** in Android:

1. Zoom Animation:

Zoom animation gives the effect of zooming in or out on a view (usually an image or layout). Achieved using **scale animation** with changing X and Y axis values.

Can be defined in **XML** or **programmatically**.

Zoom In Example (XML – `zoom_in.xml`):

xml

```
<scale xmlns:android=http://schemas.android.com/apk/res/android
        android:fromXScale="0.5"
        android:toXScale="1.0"
        android:fromYScale="0.5"
        android:toYScale="1.0"
        android:duration="500"
        android:pivotX="50%"
        android:pivotY="50%" />
```

Zoom Out Example:

xml

```
<scale xmlns:android=http://schemas.android.com/apk/res/android
        android:fromXScale="1.0"
        android:toXScale="0.5"
        android:fromYScale="1.0"
        android:toYScale="0.5"
        android:duration="500"
        android:pivotX="50%"
        android:pivotY="50%" />
```

2. Scale Animation:

A **scale animation** changes the **size (scale)** of a view.
It can be used independently or as a part of zoom effect.

In Java:

java

```
ScaleAnimation scale = new ScaleAnimation(
        1.0f, 2.0f, // fromX, toX
        1.0f, 2.0f, // fromY, toY
        Animation.RELATIVE_TO_SELF, 0.5f,
        Animation.RELATIVE_TO_SELF, 0.5f);
        scale.setDuration(500);
        view.startAnimation(scale);
```

Usage:

Both animations are used for:
Emphasizing UI elements
Image gallery effects
Interactive UI transitions

3.4 2D and 3D graphics:

Here's a concise and clear explanation on **2D and 3D Graphics in Android**:

1. 2D Graphics:

2D (Two-Dimensional) graphics deal with flat images having only height and width.
Android provides the **Canvas API** for drawing 2D shapes like lines, circles, rectangles, and text.
Can be used for custom views, games, or charts.

Key Classes:

`Canvas` – Used for drawing.
`Paint` – Defines style and color.
`Bitmap` – For image manipulation.

Example:

java

```
@Override
protected void onDraw(Canvas canvas) {
    Paint paint = new Paint();
    paint.setColor(Color.RED);
    canvas.drawCircle(100, 100, 50, paint);
}
```

2. 3D Graphics:

3D (Three-Dimensional) graphics involve width, height, and **depth (z-axis)**.
Android supports 3D graphics using **OpenGL ES (Open Graphics Library for Embedded Systems)**.
Suitable for games and simulations with complex visual effects.

Key Features:

Renders 3D objects with lighting, rotation, scaling.
More complex and powerful than 2D.
Requires knowledge of vertices, shaders, and matrices.

Example Tools:

`GLSurfaceView` – For OpenGL rendering surface.
`Renderer` – Interface for drawing 3D content.

Difference:

Feature	2D Graphics	3D Graphics
Dimensions	X, Y	X, Y, Z (depth)
Complexity	Simple	Complex (mathematics, shaders)
API Used	Canvas, Paint	OpenGL ES, GLSurfaceView
Use Case	UI, charts, simple games	Advanced games, 3D modeling